

Vivekanand Education Society's Institute Of Technology (VESIT)

Department of Information Technology

Course: Blockchain

Mini Project

Cookbook / Operations Manual

TruSecure

Decentralized Product Security using Blockchain and
Smart Contract-Based QR Verification

Roll No.	Name
45	Tanaya Jain
59	Atharva Pingale
60	Nidhi Puthran

Group No.: 11

Date: September 21, 2026

Faculty In-Charge: Mrs. Kajal Joseph

Repository: <https://github.com/tanaya2005/TruSecure-blockchain-in-Ecommerce>

1 Overview

TruSecure is a blockchain-based product authentication system that combats counterfeit goods by combining an Ethereum smart contract ledger, cryptographic hashing and QR-code verification. Every product is registered on-chain with a unique hash of its details; as the product moves through the supply chain, each stage appends an event and the hash is recalculated and checked against the original, so any tampering is instantly detectable. Consumers scan the product's QR code to independently verify authenticity without trusting any single intermediary.

This implementation reproduces the framework proposed in “*Decentralized framework for product security using blockchain and smart contract-based QR verification*” (Shivanna K. et al., *Discover Applied Sciences*, vol. 8, article 849, 2026, doi:10.1007/s42452-026-08542-z).

1.1 Roles

- **Administrator** — manages user accounts, ensuring only verified entities (manufacturers, suppliers, retailers) participate.
- **Manufacturer** — connects a MetaMask wallet, registers a product with its serial number and description, pays the registration transaction fee and receives a generated QR code.
- **Supplier** — scans the QR code to update a product's location or status during transit, confirming and paying a transaction fee for each update.
- **Retailer** — functions like the supplier, with an additional option to mark a product as sold.
- **Consumer** — scans the QR code to view the product's status; an authentic product shows its full history, a mismatch shows a counterfeit warning.

1.2 Core Stack

Layer	Tooling
Smart Contracts	Solidity, deployed on Ethereum Sepolia Testnet
Dev Environment	Hardhat (compile, test, deploy)
Blockchain Interaction	ethers.js, Node.js
Frontend	React.js, QR code scan/generate libraries
Wallet	MetaMask (Sepolia network, SepoliaETH from faucet)
Backend	Node.js + Express.js
Database	PostgreSQL (off-chain product/user metadata)
Hashing	SHA-256

2 Prerequisites

- Node.js ≥ 18 and npm
- PostgreSQL ≥ 14
- MetaMask browser extension
- A Sepolia testnet wallet funded with SepoliaETH (from a faucet)
- Git

2.1 Install Dependencies

```
git clone https://github.com/<your-username>/TruSecure.git
cd TruSecure

# smart contracts
cd contracts && npm install

# backend
cd ../backend && npm install

# frontend
cd ../frontend && npm install
```

3 Setup

3.1 1. Configure PostgreSQL

```
createdb trusecure
psql -d trusecure -f backend/schema.sql
psql -d trusecure -c "\copy products FROM 'backend/seed/products.csv' CSV HEADER"
psql -d trusecure -c "\copy users FROM 'backend/seed/users.csv' CSV HEADER"
```

Update backend/.env with the PostgreSQL credentials (DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME).

3.2 2. Deploy the Smart Contract to Sepolia

```
cd contracts
npx hardhat compile
export SEPOLIA_RPC_URL=https://rpc.sepolia.org
export PRIVATE_KEY=0x<your-metamask-private-key>
npx hardhat run scripts/deploy.js --network sepolia
```

The script prints the deployed ProductRegistry contract address and its Etherscan link. Copy the address and ABI into backend/.env and frontend/src/config.js.

3.3 3. Connect MetaMask

1. Install MetaMask and add the Sepolia test network.
2. Fund the wallet with free SepoliaETH from a faucet (e.g. <https://sepoliafaucet.com>).
3. Import this account for each role you wish to test (Manufacturer, Supplier, Retailer) — use separate accounts per role for a realistic demo.

3.4 Local-Only Fallback

If Sepolia RPC access is unavailable, run a local Hardhat network instead:

```
npx hardhat node
npx hardhat run scripts/deploy.js --network localhost
```

Then point MetaMask at <http://127.0.0.1:8545> (Chain ID 31337) and import one of the funded Hardhat test accounts.

4 Configuration

4.1 Backend Environment (backend/.env)

```
PORT=5000
DB_HOST=localhost
DB_PORT=5432
DB_USER=postgres
DB_PASSWORD=<your-password>
DB_NAME=trusecure
SEPOLIA_RPC_URL=https://rpc.sepolia.org
CONTRACT_ADDRESS=0x<deployed-address>
```

4.2 Frontend Environment (frontend/.env)

```
REACT_APP_CONTRACT_ADDRESS=0x<deployed-address>
REACT_APP_BACKEND_URL=http://localhost:5000
REACT_APP_NETWORK=sepolia
```

5 Running the System

5.1 Start Every Service Together

```
npm run dev
```

Runs the backend API and the React frontend concurrently from the project root (configured via concurrently in the root `package.json`).

5.2 Start Components Individually

```
# Terminal 1 - backend API
cd backend && npm start

# Terminal 2 - frontend
cd frontend && npm start

# open http://localhost:3000
```

6 API Reference

Endpoint	Method	Description
/auth/login	POST	Authenticates a user against their assigned role (admin issues credentials).
/products/register	POST	Manufacturer registers a product; computes $H = \text{SHA-256}(P)$, stores it on-chain and generates a QR code.
/products/update	POST	Supplier/Retailer scans a QR code and appends a supply chain event E ; contract recomputes H' .
/products/sell	POST	Retailer marks a product as sold (updates on-chain status).
/products/verify/{id}	GET	Consumer-facing check: decodes QR data, compares H' against H , returns Authentic/Compromised.
/products/{id}/history	GET	Returns the full on-chain event history of a product.

7 Blockchain Traceability Algorithm

7.1 Hash Generation

$$H = \text{Hash}(P)$$

where P is the product details (name, manufacturer, serial number) and H is a SHA-256 digest that acts as the product's digital fingerprint.

7.2 Transaction Creation

$$T_i = (P, H, T_s)$$

Each blockchain transaction stores the product details, its hash and a timestamp T_s , forming an immutable record of the product's state.

7.3 Recalculation and Comparison

$$H' = \text{Hash}(P + E), \quad \text{Verify: } H' = H$$

where E is a new supply chain event (location, movement, time). If $H' = H$ the product is unaltered; if $H' \neq H$ integrity has been breached.

7.4 Traceability Validation

$$\text{Authenticity} = \begin{cases} \text{True}, & \text{if } H' = H \\ \text{False}, & \text{otherwise} \end{cases}$$

7.5 QR Code Validation

$$D = \text{Decode}(\text{QR Code}), \quad P_{QR} = \text{Parse}(D)$$

$$\text{MatchStatus} = \begin{cases} \text{True}, & \text{if } P_{QR} = P_{\text{Blockchain}} \\ \text{False}, & \text{otherwise} \end{cases}$$

7.6 Integrated Workflow

1. Generate hash $H = \text{Hash}(P)$ at registration.
2. Store $T = (P, H, T_s)$ as a blockchain transaction.
3. On movement, update the chain: $T \leftarrow T + E$.
4. On consumer scan, recover P_{QR} and recompute $H' = \text{Hash}(P_{QR} + E)$.
5. Compare H' with H and return **Authentic** or **Compromised**.

8 Testing

```
cd contracts
npx hardhat test
```

Runs the Hardhat test suite covering product registration, event updates, hash verification and access-control restrictions per role.

8.1 End-to-End Manual Trace

1. Log in as Manufacturer (Location A) → register a product → QR code generated.
2. Log in as Supplier (Location B) → scan QR → confirm and pay to update location.
3. Log in as Retailer (Location C) → scan QR → update location, mark as available.
4. Consumer scans QR at purchase → system checks scanned location against Location C → match confirms authenticity.
5. Repeat with a copied QR code presented at a fake Location D to confirm the mismatch ($C \neq D$) is flagged as counterfeit.

9 Performance Metrics

Following the evaluation methodology of the source paper, each module is benchmarked over 1000 operations:

$$\text{Execution Time} = \frac{\text{Total Time Taken for the Operation}}{\text{Number of Operations Performed}}, \quad \text{Accuracy} = \frac{\text{Successful Operations}}{\text{Total Operations}} \times 100$$

Module	Metric Recorded	Notes
Authentication	Execution time, success rate	Login and role verification
Profile Management	Execution time, success rate	Admin account operations
Product Management	Execution time, success rate	Manufacturer registration flow
QR Code Module	Execution time, success rate	Generation and decoding
Blockchain Interaction	Execution time, latency	On-chain read/write via ethers.js
Database Operations	Execution time, success rate	PostgreSQL queries
Smart Contract Module	Execution time, gas used	Contract function calls

```
node benchmark/measure.js --operations 1000
```

Writes per-module execution time and accuracy to `benchmark/results.csv` and can be charted as execution-time-by-module, matching the source paper's evaluation.

10 Smart Contract

Contract	Public Functions
ProductRegistry	RegisterProduct(), updateEvent(), markSold(), verifyProduct() prod- ucts, records sup- ply chain events, ver- i- fies hashes

```
cd contracts
npx hardhat compile
npx hardhat test
npx hardhat run scripts/deploy.js --network sepolia
```

11 Security Threat Model

Threat	Mitigation
Counterfeit QR code copied onto a fake product	Location mismatch check ($C \neq D$) flags the product as fake on scan
Product data tampered mid-transit	Any change breaks $H' = H$ at the next verification step
Unauthorized user updates product status	Role-based access control restricts write functions to registered wallets
Centralized database single point of failure	Product hash and event history are immutable on-chain; PostgreSQL holds only off-chain metadata
Fake retailer sells counterfeit stock	Consumer verification cross-checks scanned location against the blockchain-recorded retailer location
Replay of an old QR code	Each scan re-validates against the current on-chain state, not a cached value

12 Services & Ports

Service	Port	Role	Package
Backend API	5000	Express REST API, PostgreSQL bridge, ethers.js calls	backend/
Frontend	3000	React UI — login, registration, QR scan/verify	frontend/
PostgreSQL	5432	Off-chain product/user metadata	external
Sepolia Testnet	443 (RPC)	Deployed ProductRegistry contract	external

13 Troubleshooting

Symptom	Fix
Transaction stuck / never confirms	Check SepoliaETH balance; get more from a faucet, or reset the MetaMask account nonce.
CONTRACT_ADDRESS undefined	Re-run <code>deploy.js</code> and copy the printed address into both <code>.env</code> files.
QR scan says compromised for a genuine product	Confirm the event <code>E</code> was actually recorded on-chain before the scan; check <code>/products/{id}/history</code> .
PostgreSQL connection refused	Confirm the service is running and <code>DB_*</code> variables in <code>backend/.env</code> are correct.
MetaMask shows wrong network	Manually switch to Sepolia; the frontend does not auto-switch networks.
Gas estimation failed on deploy	Increase the gas limit in <code>hardhat.config.js</code> or reduce contract complexity.

14 Repository Map

contracts/	
ProductRegistry.sol	Product registration, event log, hash verification
scripts/deploy.js	Hardhat deployment script
test/registry.test.js	Hardhat test suite
hardhat.config.js	Network and compiler configuration
backend/	
server.js	Express entry point
routes/products.js	Registration, update, verify, history endpoints
routes/auth.js	Role-based login
db.js	PostgreSQL connection pool
schema.sql	Products and users table definitions
seed/	Seed CSVs for products and users
.env	Database and contract configuration
frontend/	
src/pages/Login.jsx	Role-based login screen
src/pages/Manufacturer.jsx	Product registration and QR generation
src/pages/Supplier.jsx	QR scan and event update
src/pages/Retailer.jsx	QR scan, event update, mark as sold
src/pages/Consumer.jsx	QR scan and authenticity verification
src/config.js	Contract address and ABI
.env	Frontend environment configuration
benchmark/	
measure.js	Execution time and accuracy harness
results.csv	Recorded measurements

Reference paper: Shivanna K, Jayantkumar A. Rathod, Janardhana D R, Vidyasagar K B, Vasudev S. Shahapur, Chaitra Nayak J, “Decentralized framework for product security using blockchain and smart contract-based QR verification,” *Discover Applied Sciences*, vol. 8, article 849, 2026. <https://doi.org/10.1007/s42452-026-08542-z>